

Haya Project Report

Recommendation logic, hardware score logic, and completion summary

Project: Haya (formerly JUNE)

Purpose

Haya is an on-device AI application focused on local GGUF model discovery, device-aware recommendation, download management, model switching, and private local chat inference.

What is completed

- Device-aware recommendation engine with fastest, most recommended, and higher-quality picks.
- Hardware / compute score based on usable RAM, CPU tier, ABI, headroom, and emulator adjustment.
- Hugging Face browse, grouped variants, downloads, and import flows.
- Downloads management with active/downloaded states and use-model flow.
- Dark editorial Haya UI across hub and chat.
- Chat startup that prefers local downloaded models and opens directly into chat when available.
- Model switching and local model-name cleanup in chat.
- Shared sidebar navigation with model settings, general settings, new chat, and chat history.
- DB-backed chat history with open/delete support.
- Partial assistant output preserved when generation is stopped mid-stream.

Core architecture

- Data layer: Room database, model metadata, download state.
- Intelligence layer: DeviceProfiler, RecommendationEngine, variant scoring.
- UI layer: Model hub, downloads, import, chat, sidebar navigation.
- Runtime layer: SmolLM native GGUF inference.

Included source in this PDF

- RecommendationEngine.kt key scoring and featured selection methods.
- DeviceProfiler.kt device capability derivation.
- ModelHubScreen.kt computeDeviceScore / hardware score logic.

Recommendation entry point

E:\june2\JUNE\app\src\main\java\com\monicauditya\june\modelhub\RecommendationEngine.kt

```
56:     fun recommend(
57:         deviceProfile: DeviceProfile,
58:         models: List<ModelInfo>,
59:         userPreference: UserPreference,
60:         installedModelIds: Set<String> = emptySet(),
61:         previousFeaturedOrder: Map<String, Int> = emptyMap(),
62:     ): RecommendationResult {
63:         val profiles = models.map { model ->
64:             buildVariantProfile(deviceProfile, model, userPreference, installedModelIds)
65:         }
66:         val recommendations = profiles
67:             .map { profile ->
68:                 ModelRecommendation(
69:                     profile = profile,
70:                     isSupported = profile.estimatedRamMb <= profile.effectiveAvailableRamMb * 2
71:                 )
72:             }
73:             .sortedWith(recommendationComparator(previousFeaturedOrder))
74:
75:         return RecommendationResult(
76:             recommendations = recommendations,
77:             featured = selectFeaturedRecommendations(
78:                 recommendations = recommendations,
79:                 userPreference = userPreference,
80:                 previousFeaturedOrder = previousFeaturedOrder,
81:                 applyFirstRunBias = !userPreference.hasCompletedOnboarding ||
!userPreference.hasEditedPreferences,
82:             )
83:         )
84:     }
85:
```

Variant profile scoring

E:\june2\JUNE\app\src\main\java\com\monicauditya\june\modelhub\RecommendationEngine.kt

```
86:     private fun buildVariantProfile(
87:         deviceProfile: DeviceProfile,
88:         model: ModelInfo,
89:         userPreference: UserPreference,
90:         installedModelIds: Set<String>,
91:     ): VariantProfile {
92:         val familyCanonical = normalizeFamily(model)
93:         val subtype = resolveSubtype(model)
94:         val parameter = parseOrEstimateParameterSize(model)
95:         val quantTier = quantTierOf(model.quantization, model.fileName)
96:         val capabilities = inferCapabilities(model)
97:         val effectiveAvailableRamMb = deviceProfile.usableRamMb.coerceAtLeast(512)
98:         val weightsMb = (model.sizeGb * 1024f).toInt()
99:         val runtimeOverheadMb = (weightsMb * overheadMultiplier(quantTier)).toInt()
100:        val kvBase = kvBudgetMbPerB(userPreference.useCase)
101:        val kvCurve = (0.9f + 0.1f * log10(max(parameter.sizeB, 1f)))
102:        val kvCacheMb = (parameter.sizeB * kvBase * userPreference.sessionLengthProfile.factor *
kvCurve).toInt()
103:        val estimatedRamMb = weightsMb + runtimeOverheadMb + kvCacheMb
104:        val memoryRatio = effectiveAvailableRamMb / estimatedRamMb.toFloat()
105:        val memoryFit = memoryFit(memoryRatio)
106:        val cpuTier = deviceProfile.cpuTier
107:        val cpuFitClass = cpuFitClass(cpuTier, parameter.sizeB, quantTier)
108:        val cpuFit = cpuFit(cpuFitClass, parameter.sizeB, cpuTier)
109:        val preferenceFit = preferenceFit(userPreference, parameter.sizeB, quantTier, capabilities, memoryFit)
110:        val trustFit = trustFit(model.downloads, model.lastUpdatedEpochMs, capabilities.confidence)
111:        val latencyScore = latencyScore(estimatedRamMb, effectiveAvailableRamMb, cpuFit)
112:        val overallScoreBase = (memoryFit * 0.30f) + (cpuFit * 0.24f) + (preferenceFit * 0.36f) + (trustFit *
0.10f)
113:        val overallScore = if (installedModelIds.contains(model.id)) {
114:            (overallScoreBase + 0.05f).coerceAtMost(1.1f)
115:        } else {
116:            overallScoreBase
117:        }
118:        val performance = performanceLevel(memoryFit, cpuFit, memoryRatio)
119:        val reason = recommendationReason(memoryFit, cpuFit, preferenceFit, trustFit, userPreference,
capabilities)
120:        val whyRecommended = reasonMessage(reason)
121:
122:        return VariantProfile(
123:            model = model,
124:            familyCanonical = familyCanonical,
125:            groupCanonical =
"$familyCanonical|${parameter.sizeB.roundToIntString()}|${subtype.name.lowercase(Locale.US)}",
126:            subtype = subtype,
127:            parameterSizeB = parameter.sizeB,
128:            parameterSizeSource = parameter.source,
129:            quantTier = quantTier,
130:            capabilities = capabilities,
131:            cpuTier = cpuTier,
132:            cpuFitClass = cpuFitClass,
133:            weightsMb = weightsMb,
134:            runtimeOverheadMb = runtimeOverheadMb,
135:            kvCacheMb = kvCacheMb,
136:            estimatedRamMb = estimatedRamMb,
137:            effectiveAvailableRamMb = effectiveAvailableRamMb,
138:            memoryRatio = memoryRatio,
139:            memoryFit = memoryFit,
140:            cpuFit = cpuFit,
141:            preferenceFit = preferenceFit,
142:            trustFit = trustFit,
143:            latencyScore = latencyScore,
144:            overallScore = overallScore,
145:            performance = performance,
146:            whyRecommendedCode = reason,
```

```
147:         whyRecommended = whyRecommended,  
148:     )  
149: }  
150:
```

Featured pick selection

E:\june2\JUNE\app\src\main\java\com\monicauditya\june\modelhub\RecommendationEngine.kt

```
151:     private fun selectFeaturedRecommendations(
152:         recommendations: List<ModelRecommendation>,
153:         userPreference: UserPreference,
154:         previousFeaturedOrder: Map<String, Int>,
155:         applyFirstRunBias: Boolean,
156:     ): FeaturedRecommendations {
157:         val recommendationPool = recommendations.filter { it.profile.estimatedRamMb <=
it.profile.effectiveAvailableRamMb * 2 }
158:         val biasedPool = if (applyFirstRunBias) {
159:             recommendationPool.sortedWith(
160:                 compareBy<ModelRecommendation> { it.performance == PerformanceLevel.RISKY }
161:                     .then(recommendationComparator(previousFeaturedOrder))
162:             )
163:         } else {
164:             recommendationPool.sortedWith(recommendationComparator(previousFeaturedOrder))
165:         }
166:         val nearSafePool = biasedPool.filter {
167:             it.profile.memoryRatio >= 0.85f && it.performance != PerformanceLevel.RISKY
168:         }
169:         val safePool = biasedPool.filter { it.profile.memoryRatio >= 1.0f }
170:         val preferredPool = safePool.filter { it.profile.memoryRatio >= 1.15f }
171:         val balancedSafePool = safePool.filter { it.performance == PerformanceLevel.BALANCED }
172:         val balancedPreferredPool = preferredPool.filter { it.performance == PerformanceLevel.BALANCED }
173:         val strongMatchedBalancedSafePool = balancedSafePool.filter { isStrongUseCaseMatch(it,
userPreference.useCase) }
174:         val strongMatchedBalancedPreferredPool = balancedPreferredPool.filter { isStrongUseCaseMatch(it,
userPreference.useCase) }
175:         val matchedBalancedSafePool = balancedSafePool.filter { isUseCaseMatch(it, userPreference.useCase) }
176:         val matchedBalancedPreferredPool = balancedPreferredPool.filter { isUseCaseMatch(it,
userPreference.useCase) }
177:         val strongMatchedSafePool = safePool.filter { isStrongUseCaseMatch(it, userPreference.useCase) }
178:         val strongMatchedPreferredPool = preferredPool.filter { isStrongUseCaseMatch(it, userPreference.useCase) }
179:         val matchedSafePool = safePool.filter { isUseCaseMatch(it, userPreference.useCase) }
180:         val matchedPreferredPool = preferredPool.filter { isUseCaseMatch(it, userPreference.useCase) }
181:         val qualityCandidatePool = biasedPool.filter {
182:             it.profile.memoryRatio >= 0.78f && it.performance != PerformanceLevel.RISKY
183:         }
184:         val qualityStretchPreferredPool = preferredPool.filter {
185:             it.performance == PerformanceLevel.BALANCED || it.performance == PerformanceLevel.SLOW
186:         }
187:         val qualityStretchSafePool = qualityCandidatePool.filter {
188:             it.performance == PerformanceLevel.BALANCED || it.performance == PerformanceLevel.SLOW
189:         }
190:         val qualityStretchMatchedPreferredPool = qualityStretchPreferredPool.filter { isQualityUseCaseMatch(it,
userPreference.useCase) }
191:         val qualityStretchMatchedSafePool = qualityStretchSafePool.filter { isQualityUseCaseMatch(it,
userPreference.useCase) }
192:
193:         val chosenIds = linkedSetOf<String>()
194:         val familyCounts = mutableMapOf<String, Int>()
195:         val chosenGroups = mutableSetOf<String>()
196:         val chosenByGroup = mutableMapOf<String, ModelRecommendation>()
197:
198:         fun eligible(candidate: ModelRecommendation, allowFamilyRelaxation: Boolean = false,
allowGroupRelaxation: Boolean = false): Boolean {
199:             if (!chosenIds.add(candidate.model.id)) return false
200:             chosenIds.remove(candidate.model.id)
201:             val familyCount = familyCounts[candidate.familyCanonical] ?: 0
202:             if (!allowFamilyRelaxation && familyCount >= 2) return false
203:             if (!allowGroupRelaxation && chosenGroups.contains(candidate.groupCanonical)) return false
204:             return true
205:         }
206:
207:         fun register(candidate: ModelRecommendation?) {
```

```

208:         if (candidate == null) return
209:         if (chosenIds.contains(candidate.model.id)) return
210:         chosenIds.add(candidate.model.id)
211:         familyCounts[candidate.familyCanonical] = (familyCounts[candidate.familyCanonical] ?: 0) + 1
212:         chosenGroups.add(candidate.groupCanonical)
213:         chosenByGroup[candidate.groupCanonical] = candidate
214:     }
215:
216:     fun eligibleQuality(candidate: ModelRecommendation, allowFamilyRelaxation: Boolean = false): Boolean {
217:         if (chosenIds.contains(candidate.model.id)) return false
218:         val familyCount = familyCounts[candidate.familyCanonical] ?: 0
219:         if (!allowFamilyRelaxation && familyCount >= 2) return false
220:
221:         val existingSameGroup = chosenByGroup[candidate.groupCanonical]
222:         if (existingSameGroup != null) {
223:             return isMeaningfulQualityUpgrade(candidate, existingSameGroup)
224:         }
225:
226:         return true
227:     }
228:
229:     val fastestSelector: (List<ModelRecommendation>) -> List<ModelRecommendation> = {
230:         it.sortedWith(
231:             compareByDescending<ModelRecommendation> { it.latencyScore }
232:                 .thenBy { fastestPriority(it) }
233:                 .thenByDescending { it.overallScore }
234:                 .thenBy { previousFeaturedOrder[it.model.id] ?: Int.MAX_VALUE }
235:                 .thenBy { "it.familyCanonical|{it.groupCanonical}|it.quantTier.name|{it.model.id}" }
236:         )
237:     }
238:     val qualitySelector: (List<ModelRecommendation>) -> List<ModelRecommendation> = {
239:         it.sortedWith(
240:             compareByDescending<ModelRecommendation> { qualityRankScore(it, userPreference) }
241:                 .thenByDescending { it.overallScore }
242:                 .thenBy { previousFeaturedOrder[it.model.id] ?: Int.MAX_VALUE }
243:                 .thenBy { "it.familyCanonical|{it.groupCanonical}|it.quantTier.name|{it.model.id}" }
244:         )
245:     }
246:     val recommendedSelector: (List<ModelRecommendation>) -> List<ModelRecommendation> = {
247:         it.sortedWith(
248:             compareByDescending<ModelRecommendation> { recommendedRankScore(it, userPreference) }
249:                 .thenByDescending { it.overallScore }
250:                 .thenBy { previousFeaturedOrder[it.model.id] ?: Int.MAX_VALUE }
251:                 .thenBy { "it.familyCanonical|{it.groupCanonical}|it.quantTier.name|{it.model.id}" }
252:         )
253:     }
254:
255:     val recommended = pickCandidate(
256:         listOf(
257:             strongMatchedBalancedPreferredPool,
258:             strongMatchedBalancedSafePool,
259:             matchedBalancedPreferredPool,
260:             matchedBalancedSafePool,
261:             balancedPreferredPool,
262:             balancedSafePool,
263:             strongMatchedPreferredPool,
264:             strongMatchedSafePool,
265:             matchedPreferredPool,
266:             matchedSafePool,
267:             preferredPool,
268:             safePool,
269:             nearSafePool,
270:         ),
271:         recommendedSelector,
272:         ::eligible
273:     )
274:     register(recommended)
275:
276:     val fastest = pickCandidate(

```

```

277:         listOf(preferredPool, safePool, nearSafePool),
278:         fastestSelector,
279:         ::eligible
280:     ) ?: pickCandidate(
281:         listOf(preferredPool, safePool, nearSafePool),
282:         fastestSelector,
283:         { candidate -> eligible(candidate, allowGroupRelaxation = true) }
284:     ) ?: pickCandidate(
285:         listOf(preferredPool, safePool, nearSafePool),
286:         fastestSelector,
287:         { candidate -> eligible(candidate, allowFamilyRelaxation = true, allowGroupRelaxation = true) }
288:     ) ?: pickCandidate(
289:         listOf(preferredPool, safePool, nearSafePool),
290:         fastestSelector,
291:         { true }
292:     )
293: register(fastest)
294:
295: val qualityPools = buildQualityCandidatePools(
296:     baseline = fastest,
297:     matchedPreferred = qualityStretchMatchedPreferredPool,
298:     matchedSafe = qualityStretchMatchedSafePool,
299:     preferred = qualityStretchPreferredPool,
300:     safe = qualityStretchSafePool,
301: )
302: val highestQuality = pickCandidate(
303:     qualityPools,
304:     qualitySelector,
305:     ::eligibleQuality
306: ) ?: pickCandidate(
307:     qualityPools,
308:     qualitySelector,
309:     { candidate -> eligibleQuality(candidate, allowFamilyRelaxation = true) }
310: )
311: register(highestQuality)
312:
313: val resolvedRecommended = recommended ?: fastest
314: val resolvedFastest = fastest ?: resolvedRecommended
315: val diversifiedQuality = if (
316:     highestQuality != null &&
317:     resolvedRecommended != null &&
318:     highestQuality.familyCanonical == resolvedRecommended.familyCanonical
319: ) {
320:     qualitySelector(qualityPools.flatten().distinctBy { it.model.id }).firstOrNull { candidate ->
321:         candidate.model.id != highestQuality.model.id &&
322:         candidate.model.id != resolvedRecommended.model.id &&
323:         candidate.model.id != resolvedFastest?.model?.id &&
324:         candidate.familyCanonical != resolvedRecommended.familyCanonical &&
325:         (resolvedFastest == null || candidate.familyCanonical != resolvedFastest.familyCanonical) &&
326:         (resolvedFastest == null || isMeaningfulQualityUpgrade(candidate, resolvedFastest))
327:     } ?: highestQuality
328: } else {
329:     highestQuality
330: }
331:
332: return FeaturedRecommendations(
333:     recommended = resolvedRecommended,
334:     fastest = resolvedFastest,
335:     highestQuality = diversifiedQuality,
336: )
337: }
338:

```

Quality slot scoring

E:\june2\JUNE\app\src\main\java\com\monicauditya\june\modelhub\RecommendationEngine.kt

```
662:     private fun qualityRankScore(
663:         recommendation: ModelRecommendation,
664:         preference: UserPreference,
665:     ): Float {
666:         val capabilities = recommendation.profile.capabilities
667:         val useCaseAffinity = useCaseAffinity(capabilities, preference.useCase)
668:         val paramFit = when {
669:             recommendation.parameterSizeB >= 30f -> 1.0f
670:             recommendation.parameterSizeB >= 13f -> 0.92f
671:             recommendation.parameterSizeB >= 9f -> 0.85f
672:             recommendation.parameterSizeB >= 7f -> 0.75f
673:             recommendation.parameterSizeB >= 3f -> 0.55f
674:             else -> 0.2f
675:         }
676:         val quantFit = (quantRank(recommendation.quantTier) / 4f).coerceIn(0f, 1f)
677:         val tradeoffFit = when (recommendation.performance) {
678:             PerformanceLevel.SLOW -> 1.0f
679:             PerformanceLevel.BALANCED -> 0.88f
680:             PerformanceLevel.FAST -> 0.25f
681:             PerformanceLevel.RISKY -> 0.2f
682:         }
683:         val useCaseBias = when {
684:             isStrongUseCaseMatch(recommendation, preference.useCase) -> 0.24f
685:             useCaseAffinity >= 0.60f -> 0.12f
686:             useCaseAffinity >= 0.45f -> 0.04f
687:             else -> -0.08f
688:         }
689:         val stretchBias = when {
690:             recommendation.performance == PerformanceLevel.SLOW &&
691:                 recommendation.memoryRatio >= 0.78f -> 0.06f
692:             recommendation.performance == PerformanceLevel.BALANCED &&
693:                 recommendation.parameterSizeB >= 7f &&
694:                 recommendation.memoryRatio >= 0.90f -> 0.04f
695:             recommendation.performance == PerformanceLevel.BALANCED &&
696:                 recommendation.parameterSizeB >= 4f &&
697:                 recommendation.memoryRatio >= 0.85f -> 0.02f
698:             else -> 0f
699:         }
700:         return (
701:             (paramFit * 0.48f) +
702:             (quantFit * 0.25f) +
703:             (recommendation.profile.memoryFit * 0.10f) +
704:             (tradeoffFit * 0.17f) +
705:             useCaseBias +
706:             stretchBias
707:         ).coerceIn(0f, 1.1f)
708:     }
709:
```


Most recommended scoring

E:\june2\JUNE\app\src\main\java\com\monicauditya\june\modelhub\RecommendationEngine.kt

```
710:     private fun recommendedRankScore(
711:         recommendation: ModelRecommendation,
712:         preference: UserPreference,
713:     ): Float {
714:         val useCaseAffinity = useCaseAffinity(recommendation.profile.capabilities, preference.useCase)
715:         val useCaseBias = when {
716:             isStrongUseCaseMatch(recommendation, preference.useCase) -> 0.20f
717:             useCaseAffinity >= 0.65f -> 0.10f
718:             useCaseAffinity >= 0.45f -> 0.03f
719:             else -> -0.08f
720:         }
721:         val performanceBias = when (recommendation.performance) {
722:             PerformanceLevel.BALANCED -> 0.12f
723:             PerformanceLevel.FAST -> 0.00f
724:             PerformanceLevel.SLOW -> 0.03f
725:             PerformanceLevel.RISKY -> -0.12f
726:         }
727:         val balanceBias = when {
728:             recommendation.memoryRatio >= 1.6f &&
729:                 recommendation.parameterSizeB in 4f..9f -> 0.09f
730:             recommendation.memoryRatio >= 1.35f &&
731:                 recommendation.parameterSizeB in 3.5f..7.5f -> 0.06f
732:             recommendation.memoryRatio >= 1.15f &&
733:                 recommendation.parameterSizeB in 3.5f..7f -> 0.03f
734:             recommendation.performance == PerformanceLevel.SLOW &&
735:                 recommendation.parameterSizeB >= 4f &&
736:                 recommendation.memoryRatio >= 1.0f -> 0.04f
737:             recommendation.parameterSizeB < 2f &&
738:                 recommendation.memoryRatio >= 1.35f -> -0.07f
739:             recommendation.parameterSizeB <= 3f &&
740:                 recommendation.memoryRatio >= 1.45f -> -0.03f
741:             recommendation.parameterSizeB < 3f &&
742:                 recommendation.memoryRatio >= 1.2f -> -0.04f
743:             recommendation.performance == PerformanceLevel.FAST &&
744:                 recommendation.parameterSizeB <= 2.5f &&
745:                 recommendation.memoryRatio >= 1.35f -> -0.05f
746:             else -> 0f
747:         }
748:         val codingBias = when {
749:             preference.useCase == UserUseCase.CODING &&
750:                 recommendation.profile.capabilities.coderAffinity >= 0.45f &&
751:                 recommendation.parameterSizeB >= 3.5f &&
752:                 recommendation.memoryRatio >= 1.0f -> 0.07f
753:             preference.useCase == UserUseCase.CODING &&
754:                 recommendation.parameterSizeB < 3f &&
755:                 recommendation.memoryRatio >= 1.3f -> -0.05f
756:             else -> 0f
757:         }
758:         return (recommendation.overallScore + useCaseBias + performanceBias + balanceBias +
codingBias).coerceIn(0f, 1.25f)
759:     }
760:
```

Hardware / compute score

E:\june2\JUNE\app\src\main\java\com\monicauditya\june\ui\ModelHubScreen.kt

```
2008: private fun computeDeviceScore(deviceProfile: com.monicauditya.june.modelhub.DeviceProfile): Int {
2009:     val usableRamGb = deviceProfile.usableRamMb / 1024f
2010:     val availableRamGb = deviceProfile.availableRamMb / 1024f
2011:     val totalRamGb = deviceProfile.totalRamMb / 1024f
2012:
2013:     val ramScore = (((usableRamGb.coerceIn(2f, 16f) - 2f) / 14f) * 28f) + 18f
2014:     val headroomRatio = (deviceProfile.usableRamMb.toFloat() /
deviceProfile.availableRamMb.coerceAtLeast(1)).coerceIn(0.30f, 1.0f)
2015:     val headroomScore = ((headroomRatio - 0.30f) / 0.70f) * 10f
2016:     val cpuTierScore = when (deviceProfile.cpuTier) {
2017:         com.monicauditya.june.modelhub.CpuTier.HIGH -> 30f
2018:         com.monicauditya.june.modelhub.CpuTier.MID -> 22f
2019:         com.monicauditya.june.modelhub.CpuTier.LOW -> 14f
2020:     }
2021:     val cpuCoreBonus = ((deviceProfile.cpuCores - 4).coerceIn(0, 8) * 1.5f)
2022:     val abiScore = if (deviceProfile.abi.contains("arm64", ignoreCase = true)) 8f else 5f
2023:     val capacityBonus = ((availableRamGb / totalRamGb.coerceAtLeast(1f)) * 4f).coerceIn(0f, 4f)
2024:     val emulatorPenalty = if (deviceProfile.isEmulator) 6f else 0f
2025:
2026:     return (ramScore + headroomScore + cpuTierScore + cpuCoreBonus + abiScore + capacityBonus - emulatorPenalty)
2027:         .toInt()
2028:         .coerceIn(20, 100)
2029: }
2030:
```

Device profile data

E:\june2\JUNE\app\src\main\java\com\monicauditya\june\modelhub\DeviceProfiler.kt

```
8: data class DeviceProfile(  
9:     val totalRamMb: Int,  
10:    val availableRamMb: Int,  
11:    val usableRamMb: Int,  
12:    val cpuCores: Int,  
13:    val cpuTier: CpuTier,  
14:    val abi: String,  
15:    val isEmulator: Boolean,  
16: )  
17:
```

Device profiling and CPU tiering

E:\june2\JUNE\app\src\main\java\com\monicauditya\june\modelhub\DeviceProfiler.kt

```
19: class DeviceProfiler(private val context: Context) {
20:
21:     fun getDeviceProfile(): DeviceProfile {
22:         val activityManager = context.getSystemService(Context.ACTIVITY_SERVICE) as ActivityManager
23:         val memoryInfo = ActivityManager.MemoryInfo().also(activityManager::getMemoryInfo)
24:         val totalRamMb = bytesToMb(memoryInfo.totalMem)
25:         val availableRamMb = bytesToMb(memoryInfo.availMem)
26:         val cpuCores = Runtime.getRuntime().availableProcessors()
27:         val abi = Build.SUPPORTED_ABIS.firstOrNull().orEmpty()
28:         val isEmulator = isProbablyEmulator()
29:         val usableRamMb = computeUsableRam(
30:             totalRamMb = totalRamMb,
31:             availableRamMb = availableRamMb,
32:             isEmulator = isEmulator,
33:         )
34:
35:         return DeviceProfile(
36:             totalRamMb = totalRamMb,
37:             availableRamMb = availableRamMb,
38:             usableRamMb = usableRamMb,
39:             cpuCores = cpuCores,
40:             cpuTier = classifyCpuTier(
41:                 cpuCores = cpuCores,
42:                 abi = abi,
43:                 usableRamMb = usableRamMb,
44:                 isEmulator = isEmulator,
45:             ),
46:             abi = abi,
47:             isEmulator = isEmulator
48:         )
49:     }
50:
51:     private fun bytesToMb(bytes: Long): Int = (bytes / (1024L * 1024L)).toInt()
52:
53:     private fun computeUsableRam(
54:         totalRamMb: Int,
55:         availableRamMb: Int,
56:         isEmulator: Boolean,
57:     ): Int {
58:         val reserveMb = when {
59:             isEmulator && availableRamMb >= 12 * 1024 -> 1024
60:             isEmulator -> 768
61:             totalRamMb >= 16 * 1024 -> 1280
62:             totalRamMb >= 8 * 1024 -> 896
63:             totalRamMb >= 4 * 1024 -> 640
64:             else -> 448
65:         }
66:         return (availableRamMb - reserveMb).coerceAtLeast(512)
67:     }
68:
69:     private fun classifyCpuTier(
70:         cpuCores: Int,
71:         abi: String,
72:         usableRamMb: Int,
73:         isEmulator: Boolean,
74:     ): CpuTier {
75:         var tier = when {
76:             cpuCores <= 4 -> CpuTier.LOW
77:             cpuCores <= 8 -> CpuTier.MID
78:             else -> CpuTier.HIGH
79:         }
80:
81:         if (!isEmulator && !abi.contains("arm64", ignoreCase = true)) {
82:             tier = downgrade(tier)
83:         }
84:     }
85: }
```

```
84:
85:     if (usableRamMb < 4096) {
86:         tier = downgrade(tier)
87:     }
88:
89:     if (isEmulator && abi.contains("x86_64", ignoreCase = true) && usableRamMb >= 8192 && tier ==
CpuTier.LOW) {
90:         tier = CpuTier.MID
91:     }
92:
93:     return tier
94: }
95:
```